

J-ART: A Japanese Adversarial Red-Team Framework for Application-Layer LLM Security and Cost-Efficiency

Technical Report (v0.1, preprint)

Author: Takayuki Hirose — ORCID: [0009-0005-6735-1862](https://orcid.org/0009-0005-6735-1862) Affiliation: Independent Researcher Project / code: <https://github.com/takker-hero-se/J-ART> Live leaderboard: <https://takker-hero-se.github.io/J-ART/> DOI (concept): [10.5281/zenodo.20676879](https://doi.org/10.5281/zenodo.20676879) — v0.1.0: 10.5281/zenodo.20676880 License: report text CC-BY-4.0; code MIT.

*Status. This is a preliminary technical report / preprint describing a proof-of-concept framework and exploratory findings. It is **not** an official safety evaluation of any vendor's model. Numbers are run-time snapshots and are subject to the limitations in §7. References (§10) are indicative and their bibliographic details should be verified before formal submission.*

Abstract

Most open red-teaming and jailbreak benchmarks for large language models (LLMs) are English-centric and evaluate *bare models*. We present **J-ART** (Japanese Adversarial Red-Team framework), an open, reproducible harness that (i) evaluates the **application-structure layer** — model × system-prompt strength × guardrail × retrieval (RAG) — rather than bare models; (ii) applies **Japanese-specific obfuscation transforms** (gyaru-moji character substitution, vertical/newline splitting, keigo "polite-coercion" framing, double-talk false premises, Base64, and leet/zero-width smuggling) that defeat naïve keyword filters; (iii) grounds attacks in **MITRE ATLAS** (10 inference-time techniques); and (iv) reports a **cost-efficiency** metric alongside a defense rate with **Wilson 95% confidence intervals**. Attacks use *harmless proxy markers* (a fictitious canary and benign output markers) so the framework contains **no weaponizable content**, and the malicious "core" of each attack is masked in all published artifacts.

Across 18–21 application configurations and 11 attacks × 7 transforms, we find that **application-layer defenses dominate raw model capability** (a flagship model with no system prompt or guardrail was breached more often than a small, cheap model behind a hardened prompt and a keyword guardrail), that a dedicated **input classifier (Llama Guard)** eliminated all observed breaches for a weak configuration at lower net cost, and that a simple **normalizing filter** defeats the obfuscation transforms that a naïve keyword filter misses. We also document **large run-to-run variance** for identical configurations served via a model router (one configuration moved from 58 breaches to 0 breaches between two runs), which we argue makes single-sample leaderboards unreliable and motivates the statistical treatment we adopt.

概要（日本語）

LLM のレッドチームing/ジェイルブレイク評価はほぼ英語中心で、かつ「素のモデル」を対象とする。本稿は **J-ART**（日本語敵対的レッドチーム評価ハネス）を提案する。特徴は、(1)「モデル × システムプロンプト強度 × ガードレール × RAG」という **アプリケーション構造層** を評価すること、(2) ギャル文字・縦書き改行・慇懃無礼・二枚舌・Base64・leet/ゼロ幅密輸といった **日本語特有の難読化変形** でキーワード検閲を突破すること、(3) 攻撃を **MITRE ATLAS**（推論時10技術）に対応づけること、(4) 防御率に **Wilson 95%信頼区間** を付し、**コスト効率指標** を併記すること、である。攻撃は無害なプロキシ（架空カナリア・無害マーカー）で構成され、危険物を一切含まず、悪意あるコアは公開物上で常にマスクされる。実験では、**アプリ層の防御がモデル素性を上回る**こと、**専用入力分類器（Llama Guard）が弱構成の突破を全て遮断**したこと、**正規化フィルタが難読化を無効化**することを示す。さらに、同一構成でも実行間で大きくばらつく（ある構成が58突破→0突破に変動）ことを記録し、単一サンプルのリーダーボードが信頼できないこと、統計的処理が必要であることを論じる。

1. Introduction

Public LLM red-teaming tooling (e.g., garak) and jailbreak benchmarks (e.g., HarmBench, JailbreakBench) are predominantly English. Multilingual safety work has shown that *translating* prompts into low-resource languages can bypass safety training, but **language-specific obfuscation** — exploiting a language's own scripts, typography, and register — remains comparatively underexplored, and Japanese in particular offers a rich obfuscation surface (multiple scripts, full-width/zero-width characters, vertical writing, and highly conventionalized polite registers).

Two further gaps motivate J-ART:

1. **Bare-model vs. deployed-system evaluation.** Practitioners do not deploy bare models; they deploy a model *plus* a system prompt, *plus* one or more guardrails, often *plus* retrieval. The security-relevant unit is the **application configuration**, not the model. J-ART evaluates configurations.
2. **Security *and* cost, jointly.** Defenses cost tokens (LLM-as-judge guardrails) or latency. We report a **cost-efficiency** score so that "cheapest sufficiently-safe configuration" is directly readable.

Our contributions are: (a) a Japanese obfuscation transform suite; (b) an application-layer, MITRE-ATLAS-grounded evaluation harness with a safe proxy-marker methodology; (c) a guardrail comparison including a normalizing filter and a model-based classifier; (d) a statistically-reported leaderboard (Wilson CIs, optional repeated trials) with a cost-efficiency metric; and (e) an empirical observation of large router-induced run-to-run variance.

2. Related Work

- **Red-teaming tooling:** garak (LLM vulnerability scanner).
- **Jailbreak benchmarks:** HarmBench; JailbreakBench; AdvBench.
- **Automated attack generation:** PAIR; TAP.
- **Multilingual safety:** low-resource-language jailbreaks of aligned models.
- **Indirect prompt injection:** Greshake et al., "Not what you've signed up for."
- **Guardrails / classifiers:** Llama Guard; OpenAI Moderation.
- **Taxonomy & checklists:** MITRE ATLAS; OWASP Top 10 for LLM Applications.

J-ART differs by combining a *Japanese obfuscation* surface, an *application-layer* unit of evaluation, a *cost-efficiency* axis, and a *safe proxy-marker* methodology in a single reproducible harness. (*Bibliographic details to be finalized; see §10.*)

3. Methodology

3.1 Application-structure layer

Each evaluated **target** is a configuration: `provider/model × prompt_strength ∈ {naked(low), hardened(high)} × guardrail ∈ {none, keyword, regex, llm, llamaguard} × rag ∈ {true, false}`. A simplified RAG context (benign corporate documents, optionally poisoned for indirect-injection attacks) and a system prompt containing a secret **canary** are assembled per trial.

3.2 Japanese obfuscation transforms

Each base attack is rendered through one of seven transforms: `baseline` (none), `polite_business` (keigo authority framing), `vertical_newline` (one-character-per-line vertical writing), `gyaru` (gyaru-moji glyph substitution + zero-width spaces), `double_tongue` (false sandbox/authorization premise), `base64_wrap` (Base64-encoded instruction), and `leet_smuggle` (leet symbol substitution + zero-width spaces). Transforms are designed to preserve human legibility while breaking substring-matching filters.

3.3 Attack suite and MITRE ATLAS mapping

Eleven attacks cover ten inference-time ATLAS techniques: Direct/Indirect Prompt Injection (AML.T0051.000/.001), LLM Jailbreak (AML.T0054), LLM Data Leakage (AML.T0057), LLM Prompt Self-Replication (AML.T0061), Trusted-Output/Citations manipulation (AML.T0067.000), LLM Prompt Obfuscation (AML.T0068), System-Prompt discovery (AML.T0069.002), RAG Poisoning (AML.T0070), and False RAG Entry Injection (AML.T0071). Vectors are `user` (direct) or `rag` (indirect).

3.4 Harmless proxy markers and safety-by-construction

To avoid producing weaponizable content, attacks target **harmless proxies**: a fictitious canary passphrase embedded in the system prompt (data-leakage attacks) and benign string markers `[JART-PWNED]` / `[JART-HIJACKED]` (injection / control-hijack attacks). The malicious instruction **core** of each attack is defined in code, kept out of configuration files, and **masked** (the core is replaced by a redaction placeholder) in `results.json` and the public site; only the obfuscation *framework* is shown, so developers can study the structure without a usable recipe. A defense-in-depth client-side mask is also applied at render time.

3.5 Deterministic judge

A breach is declared by exact membership: the canary in the response (`secret_leak`), or the marker in the response (`marker_injection` / `marker_hijack`). This judge is reproducible and cheap but conservative (it can miss semantically-equivalent leaks); see §7.

3.6 Guardrails

Five guardrail settings are compared: **none**; **keyword** (banned-substring match — fast, weak to obfuscation); **regex** (a *normalizing* filter that strips zero-width characters, reverses leet substitutions, folds whitespace, and Base64-decodes long tokens *before* keyword matching — deterministic, zero extra API cost); **llm** (the target model self-moderates input, and additionally screens output); and **llamaguard** (a dedicated safety-classification model — Llama Guard 4 via a model router — classifies the input as attack/benign, with a graceful fallback on failure).

3.7 Statistical treatment

Each model's defense rate is reported with a **Wilson 95% confidence interval** computed over its trial count. Optionally, each cell (target, attack, transform) is repeated `N` times (`JART_TRIALS` , default 1); details aggregate per cell with a per-cell breach rate, while summary statistics and CIs use the trial-level sample. In simulation mode each repeated trial is an independent Bernoulli draw, which (as a sanity check) narrows the CI as `N` grows (e.g., interval width 15.1 → 7.5 points from `N` = 1 to `N` = 3 for one configuration).

3.8 Cost-efficiency metric

We report $\text{cospa} = \text{defense_rate}(\%) \div \text{cost_per_million_tokens}(\text{USD})$ (the per-million cost is clamped at a 0.01 USD floor to avoid divergence). This surfaces configurations that are both safe and cheap.

3.9 Execution

Trials are independent and are executed with bounded concurrency (`ThreadPoolExecutor` , default 8 workers) under per-request timeouts and exponential-backoff retries. Determinism of the simulation does not depend on execution order. Targets whose API key is absent fall back to a deterministic simulation so a partial key set never aborts the run.

4. Results

We summarize two live runs (run A: 18 configurations, 1,386 cells; run B: 21 configurations, 1,617 cells), one trial per cell.

4.1 Overall

Breach rates were low in aggregate (run A: $108/1,386 = 7.8\%$; run B: $36/1,617 = 2.2\%$) and concentrated in **weak configurations** (naked, low-strength models), while hardened-prompt-plus-guardrail configurations defended at or near 100%.

4.2 Application-layer defense dominates model capability

In run A, a flagship model with **no system prompt and no guardrail** was breached 10 times (87.0% defense), whereas a small, inexpensive model behind a **hardened prompt + keyword guardrail** defended at 96%+. This is the central practical finding: *how the application is configured matters more than which model is used*.

4.3 Run-to-run variance (a primary result)

The **identical** configuration "Llama 4 Scout / naked" produced **58 breaches (24.7% defense)** in run A and **0 breaches (100% defense)** in run B. Because the only change between runs was time of execution (the model was served via a router that may dispatch to different backends/quantizations), this demonstrates that **single-sample LLM leaderboards are not reproducible**, and that defense rates should be reported with confidence intervals and, ideally, repeated trials. We treat this variance as a result, not noise to be hidden.

4.4 Guardrail comparison

In run B, "GPT-4o-mini / naked" was breached 14 times (81.8% defense, CI[71.8–88.8]); the **same model behind a Llama-Guard input classifier** was breached **0 times** (100% defense, CI[95.2–100]) at lower net cost, because blocked inputs skip the main model call. The **normalizing regex filter**, in unit tests, deterministically neutralizes the vertical-newline, leet, Base64, and gyaru transforms that the keyword filter misses, at zero additional API cost; in run B its weak-model baselines did not breach, so its live preventive effect was not separable in that snapshot (see §7).

4.5 Attack and transform effectiveness

In run A, the most effective attacks were Trusted-Output/Citations manipulation, False RAG Entry Injection, and indirect RAG exfiltration; the most effective transforms were vertical-newline, baseline, and polite-business, while **Base64** was least effective (models frequently declined to act on encoded instructions). The newly-added ATLAS techniques accounted for a large share of breaches, indicating the expanded taxonomy contributes signal.

5. The Leaderboard Artifact

The harness emits `results.json` and a self-contained static site (sortable leaderboard, per-configuration attack logs with masked prompts, JP/EN UI with environment-based auto-selection, per-model defense-rate CIs, and per-cell breach rates). The site is continuously deployed.

6. Ethics and Responsible Disclosure

J-ART is designed to be safe by construction. (1) Attacks target a **fictitious canary** and **benign markers**, never real harmful capabilities; no weaponizable content exists in the code, the data, or the published site. (2) The malicious instruction **core** is masked in all artifacts; only the obfuscation framework is shown. (3) Models are referred to for research comparison only; results are run-time snapshots, not certifications. The intended use is to help developers harden their *own* deployments.

7. Limitations

- **Proxy markers, not harm:** we measure instruction-violation / leakage proxies rather than real-world harm.

- **Exact-match judge:** semantic or transformed leaks may be undercounted; a validated LLM-judge (with human agreement) is future work.
- **Single-turn:** multi-turn / crescendo attacks are not yet modeled.
- **Sample size:** a handcrafted suite (11 attacks × 7 transforms); default `N = 1` per cell.
- **Provider non-determinism / router variance:** see §4.3; results are snapshots and depend on routing and model versions at run time.
- **Cost attribution:** guardrail tokens are attributed at the target model's price (an approximation).
- **Simulation mode** is a deterministic stand-in for missing keys and must not be read as empirical model behavior.

8. Future Work

LLM-judge with human-validated agreement; multi-turn and adaptive (PAIR/TAP-style) attacks; larger and partially human-curated attack sets; additional ATLAS techniques (cost-exhaustion, model extraction); per-provider concurrency and incremental caching with longitudinal trend tracking; and dropping simulation mode for a purely empirical, repeated-trial statistical report.

9. Reproducibility

Code, configuration, and the deployment workflow are public. The harness is deterministic in simulation mode; live results depend on provider routing and model versions at run time and should be reported with the run date and model identifiers. See the repository for exact model identifiers and pricing used.

10. References (indicative — verify before submission)

1. MITRE ATLAS — Adversarial Threat Landscape for AI Systems. atlas.mitre.org
2. garak — Generative AI Red-teaming & Assessment Kit (NVIDIA).
3. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming.
4. JailbreakBench: An Open Robustness Benchmark for Jailbreaking LLMs.
5. Yong, Menghini, Bach. Low-Resource Languages Jailbreak GPT-4 (2023).
6. Chao et al. Jailbreaking Black-Box LLMs in Twenty Queries (PAIR).
7. Mehrotra et al. Tree of Attacks: Jailbreaking Black-Box LLMs Automatically (TAP).
8. Greshake et al. Not What You've Signed Up For: Indirect Prompt Injection (2023).
9. Inan et al. Llama Guard (Meta, 2023).
10. OWASP Top 10 for Large Language Model Applications.

Citation

If you use J-ART, please cite the archived release (see `CITATION.cff` / `.zenodo.json` in the repository; a DOI is minted on first Zenodo release).